

BÀI THỰC HÀNH

LAB 02

Cơ bản về Digital Input

Đọc nút nhấn và điều khiển GPIO với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Digital Input và mức logic	2
2.2	Nút nhấn như một Digital Input	2
2.3	LED ngoài và điện trở hạn dòng	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Ánh xạ chân	3
3.3	Kết nối LED ngoài	3
4	EduFramework API	4
4.1	<code>pinMode()</code> với <code>INPUT</code>	4
4.2	<code>digitalRead()</code>	4
4.3	<code>digitalToggle()</code>	5
5	Bài thực hành	5
5.1	Yêu cầu	5
5.2	Chương trình	5
5.3	Kiểm chứng	6
6	Mở rộng	6
6.1	Nút nhấn ngoài và điện trở kéo	6
6.2	Button debouncing	7
7	Tài liệu tham khảo	7

1 Giới thiệu

1.1 Tổng quan bài lab

Digital Input cho phép vi điều khiển nhận trạng thái logic từ các tín hiệu bên ngoài thông qua GPIO. Khi một chân được cấu hình làm đầu vào số, chương trình có thể đọc trạng thái tại chân và sử dụng kết quả đó để thực hiện các hành vi điều khiển tương ứng.

Bài lab này được thiết kế để giới thiệu Digital Input thông qua một nút nhấn và các API của EduFramework. Trạng thái của nút nhấn được sử dụng để điều khiển một LED ngoài kết nối với GPIO.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Giải thích nguyên lý cơ bản của Digital Input và các trạng thái logic.
2. Sử dụng EduFramework để cấu hình và đọc trạng thái của nút nhấn thông qua GPIO.
3. Kết nối và điều khiển LED ngoài bằng GPIO.
4. Xây dựng và kiểm chứng ứng dụng sử dụng nút nhấn để thay đổi trạng thái của LED.

2 Kiến thức nền

2.1 Digital Input và mức logic

GPIO (General-Purpose Input/Output) có thể được cấu hình làm đầu vào hoặc đầu ra số. Khi được cấu hình làm Digital Input, chân GPIO được sử dụng để đọc mức logic của tín hiệu bên ngoài.

Trong EduFramework, trạng thái Digital Input được biểu diễn bằng `LOW` và `HIGH`.

Trạng thái	Giá trị	Ý nghĩa
<code>LOW</code>	<code>0U</code>	Mức logic thấp được phát hiện tại Digital Input.
<code>HIGH</code>	<code>1U</code>	Mức logic cao được phát hiện tại Digital Input.

1. `"-" + str(lab-number)`
2. `"-" + str(section-number)`

Bảng 2.2.1. Trạng thái logic của Digital Input

`HIGH` và `LOW` không biểu diễn một giá trị điện áp cố định. Ngưỡng điện áp được nhận biết là logic cao hoặc logic thấp phụ thuộc vào đặc tính điện của vi điều khiển. Các giới hạn đối với S32K1xx được quy định trong Data Sheet của NXP [2].

2.2 Nút nhấn như một Digital Input

Nút nhấn có hai trạng thái cơ bản: nhấn và thả. Khi thao tác nút làm thay đổi mức logic tại Digital Input, chương trình có thể đọc mức này để xác định trạng thái hiện tại của nút.

Trong một ứng dụng điển hình, Digital Input có thể được đọc liên tục trong vòng lặp chính để chương trình phản hồi khi trạng thái của tín hiệu thay đổi.

2.3 LED ngoài và điện trở hạn dòng

LED (Light-Emitting Diode) là linh kiện có cực tính với hai cực anode và cathode. Khi LED được điều khiển từ GPIO, điện trở mắc nối tiếp được sử dụng để giới hạn dòng qua LED [4] .

Khi kết nối LED ngoài với board phát triển, cần bảo đảm đúng cực tính của LED và sử dụng chung GND giữa mạch ngoài và board.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
LED	LED ngoài được sử dụng làm đầu ra trực quan.
Điện trở 330 Ω	Điện trở mắc nối tiếp để giới hạn dòng qua LED.
Breadboard	Sử dụng để lắp LED và điện trở.
Jumper wires	Sử dụng để kết nối mạch ngoài với board phát triển.

1. “-” + str(lab-number)
2. “-” + str(section-number)

Bảng 2.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân

Bài thực hành sử dụng nút nhấn BTN0 tích hợp trên board làm Digital Input và Logical Pin GPIO0 làm Digital Output. Ánh xạ phần cứng được trình bày trong bảng dưới đây [3] .

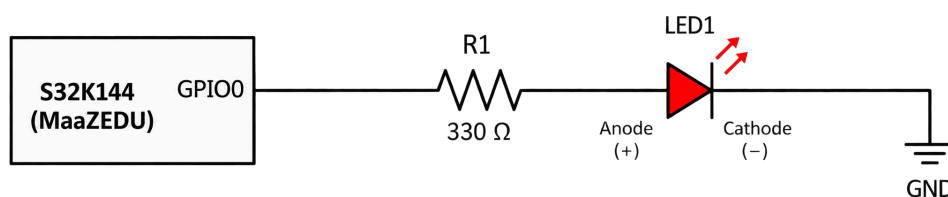
Thành phần	Logical Pin	MCU Pin	Chức năng
Nút nhấn tích hợp	BTN0	PTC12	Digital Input
LED ngoài	GPIO0	PTE0	Digital Output

1. “-” + str(lab-number)
2. “-” + str(section-number)

Bảng 2.3.2. Ánh xạ Digital I/O sử dụng trong bài thực hành

3.3 Kết nối LED ngoài

LED ngoài được mắc nối tiếp với điện trở hạn dòng giữa GPIO0 và GND. Sơ đồ kết nối được trình bày trong hình dưới đây.



1. "-" + str(lab-number)
2. "-" + str(section-number)

Hình 2.3.1. Sơ đồ kết nối LED ngoài với GPIO

Ghi chú

LED là linh kiện có cực tính. Cần xác định đúng anode và cathode trước khi cấp nguồn cho mạch.

4 EduFramework API

Bài lab sử dụng các API Digital I/O của EduFramework để cấu hình đầu vào, đọc trạng thái logic và thay đổi trạng thái của Digital Output.

4.1 pinMode() với INPUT

`pinMode()` cấu hình chế độ hoạt động của một Logical Pin. Trong bài lab này, chế độ `INPUT` được sử dụng để cấu hình chân làm Digital Input.

pinMode

Cấu hình Logical Pin được chỉ định làm Digital Input.

Cú pháp

```
pinMode(pin, INPUT);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	Logical Pin	Chân cần cấu hình làm đầu vào số.
<code>mode</code>	<code>INPUT</code>	Chế độ Digital Input.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
pinMode(BTN0, INPUT);
```

4.2 digitalRead()

`digitalRead()` được sử dụng để đọc trạng thái logic hiện tại của một Logical Pin [1].

digitalRead

Đọc trạng thái logic hiện tại của Logical Pin được chỉ định.

Cú pháp

```
digitalRead(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	Logical Pin	Chân Digital Input cần đọc trạng thái.

Giá trị trả về

HIGH khi mức logic cao được phát hiện; **LOW** khi mức logic thấp được phát hiện.

Ví dụ:

```
if (HIGH == digitalRead(BTN0))
{
    /* Button is pressed */
}
```

4.3 digitalToggle()

digitalToggle() đảo mức logic hiện tại của một Digital Output.

digitalToggle

Đảo trạng thái logic hiện tại của Logical Pin được chỉ định.

Cú pháp

```
digitalToggle(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
pin	Logical Pin	Chân Digital Output cần đảo trạng thái.

Giá trị trả về

Không trả về giá trị.

Ví dụ:

```
digitalToggle(GPIO0);
```

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình sử dụng **BTN0** để điều khiển LED ngoài kết nối với **GPIO0**. Chương trình cần đáp ứng các yêu cầu sau:

- LED ngoài ban đầu ở trạng thái tắt.
- Mỗi lần **BTN0** được nhấn, trạng thái LED được đảo một lần.
- Giữ **BTN0** không làm LED thay đổi trạng thái liên tục.
- Sau khi thả **BTN0**, chương trình sẵn sàng nhận lần nhấn tiếp theo.

5.2 Chương trình

Trong **src/main.c**, triển khai chương trình như sau:

```
#include "Arduino.h"

int main(void)
{
    bool pressed = false;
```

```
setup();

pinMode(BTN0, INPUT);
pinMode(GPIO0, OUTPUT);

digitalWrite(GPIO0, LOW);

while (1)
{
    if ((HIGH == digitalRead(BTN0)) && (false == pressed))
    {
        digitalWrite(GPIO0);
        pressed = true;
    }

    if (LOW == digitalRead(BTN0))
    {
        pressed = false;
    }
}

return 0;
}
```

1. "-" + str(lab-number)
2. "-" + str(section-number)

Mã nguồn 2.5.1. Chương trình điều khiển LED ngoài bằng nút nhấn

BTN0 được cấu hình làm Digital Input và GPIO0 làm Digital Output. LED được đặt ở trạng thái ban đầu bằng `digitalWrite(GPIO0, LOW)`.

Biến `pressed` được sử dụng để ghi nhận lần nhấn hiện tại đã được xử lý hay chưa. Khi BTN0 ở mức HIGH và `pressed` bằng `false`, chương trình gọi `digitalToggle()` để đảo trạng thái LED và đặt `pressed` thành `true`. Vì vậy, việc tiếp tục giữ nút không làm LED bị đảo trạng thái nhiều lần.

Khi nút được thả và `digitalRead(BTN0)` trả về LOW, `pressed` được đặt lại thành `false`. Chương trình sau đó có thể xử lý lần nhấn tiếp theo.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board. Quan sát LED ngoài trong quá trình nhấn, giữ và thả BTN0.

KẾT QUẢ MONG ĐỢI

LED ngoài ban đầu tắt. Mỗi lần BTN0 được nhấn, LED đổi trạng thái đúng một lần. Việc giữ nút không làm LED thay đổi liên tục. Sau khi thả nút, lần nhấn tiếp theo tiếp tục đảo trạng thái LED.

6 Mở rộng

6.1 Nút nhấn ngoài và điện trở kéo

Khi một Digital Input không được chủ động nối tới mức logic cao hoặc thấp, trạng thái tại chân có thể không được xác định ổn định. Điện trở pull-up hoặc pull-down được sử dụng để thiết lập trạng thái mặc định cho input khi công tắc đang mở.

S32K1xx tích hợp điện trở pull-up và pull-down cho các chân I/O. Trong điều kiện I/O 3.3 V, Data Sheet quy định điện trở kéo nội có dải từ 20 kΩ đến 60 kΩ [2].

EduFramework cung cấp hai chế độ Digital Input có điện trở kéo:

Chế độ	Chức năng
INPUT_PULLUP	Digital Input với điện trở kéo lên nội.
INPUT_PULLDOWN	Digital Input với điện trở kéo xuống nội.

1. “-” + str(lab-number)
2. “-” + str(section-number)

Bảng 2.6.1. Các chế độ Digital Input có điện trở kéo

Ví dụ:

```
pinMode(pin, INPUT_PULLUP);  
pinMode(pin, INPUT_PULLDOWN);
```

Mở rộng: Thay nút nhấn tích hợp bằng một nút nhấn ngoài kết nối với một Logical Pin GPIO khác. Lựa chọn INPUT_PULLUP hoặc INPUT_PULLDOWN phù hợp với cách đấu mạch và điều chỉnh điều kiện phát hiện trạng thái nhấn.

6.2 Button debouncing

Tiếp điểm cơ học của nút nhấn có thể tạo nhiều chuyển mức ngắn trong quá trình đóng hoặc mở. Hiện tượng này được gọi là contact bounce và có thể khiến một thao tác vật lý được nhận thành nhiều sự kiện số [5].

Debouncing là quá trình hạn chế các chuyển mức không mong muốn này. Tùy yêu cầu của ứng dụng, debouncing có thể được thực hiện bằng phần cứng hoặc phần mềm.

EduFramework tự động bật passive input filter cho các Logical Pin được xác định là nút nhấn tích hợp. Cơ chế này giúp lọc các xung ngắn tại input nhưng không thay thế một phương pháp debounce hoàn chỉnh trong mọi trường hợp.

Mở rộng: Sử dụng nút nhấn ngoài và đề xuất một phương pháp software debounce để chỉ xử lý trạng thái nút sau khi tín hiệu đã ổn định trong một khoảng thời gian xác định.

7 Tài liệu tham khảo

- [1] Arduino, *digitalRead()*, Arduino Language Reference.
- [2] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [3] MaaZEDU, *MaaZEDU Development Board Guide*.
- [4] University of Wisconsin-Madison, *GPIO Pins*, ECE353 - Introduction to Microprocessor Systems.
- [5] Texas Instruments, *Debounce a Switch*, SCEA094.